E3 — Testing con pytest sobre el pipeline de ventas

Explicación de la práctica y de los conceptos teóricos para repasar de cara al examen.

1. Qué se ha hecho en la práctica

Objetivo: escribir **tests automáticos** que comprueben que el paquete `ventas_app` de la E2 funciona, tanto con datos pequeños en memoria como con el CSV real del curso.

Estructura dentro de `1_programacion_avanzada_python/`:

```
1_programacion_avanzada_python/
  ventas_app/            # código que se prueba (E2)
  tests/
    test_validator.py    # fixture, parametrize, raises
    test_loader.py       # carga CSV/JSON, errores, tmp_path
    test_metrics.py      # agregaciones con datos en memoria
    test_integration.py  # ventas.csv real (marca integration)
  pytest.ini             # configuración de pytest
```

Parte	Qué se hace	Herramienta clave
0. Calentamiento	Ejecutar los tests de ejemplo del profesor (`calculator`) y romper uno a propósito para leer el informe de fallo	`pytest -q`
1. Fixture	Datos en memoria (3 filas: 1 válida, 2 inválidas) y tests del número de válidas/inválidas	`@pytest.fixture`
2. Errores y bordes	Error si el fichero no existe; precios límite `0`, `-1`, `1`	`pytest.raises`, `@pytest.mark.parametrize`
3. Integración	El `ventas.csv` real da 140 válidas y 10 inválidas	`@pytest.mark.integration`, `-m`
4. Documentar	Comandos de test en el README	—

Resultado:

```
pytest -q                        ->  20 passed
pytest -q -m "not integration"   ->  18 passed, 2 deselected
```

20 tests (el enunciado pide al menos 8), de los cuales 2 son de integración.

Tests por fichero:

`test_validator.py` (9 tests)

Test	Qué comprueba	Concepto
`test_validar_ventas_mini_una_valida`	La fixture produce 1 fila válida	fixture + AAA
`test_validar_ventas_mini_dos_invalidas`	La fixture produce 2 filas inválidas	fixture
`test_validar_ventas_importe_es_unidades_por_precio`	importe = 2 × 10.0 = 20.0	columna derivada
`test_validar_ventas_no_modifica_el_original`	`.copy()` protege el DataFrame original	sin efectos laterales
`test_validar_ventas_precio_limite` (×3)	Precio 0 y -1 inválidos, 1 válido	parametrize + bordes
`test_validar_ventas_unidades_texto_va_a_errores`	`"na"` en unidades → fila inválida	borde de tipo
`test_validar_ventas_sin_columna_obligatoria_lanza_validation_error`	Falta una columna → `ValidationError`	raises

`test_loader.py` (5 tests)

Test	Qué comprueba	Concepto
`test_load_path_inexistente_lanza_data_load_error`	Fichero inexistente → `DataLoadError`	raises
`test_load_formato_no_soportado_lanza_data_load_error`	Extensión `.txt` → `DataLoadError`	raises + `tmp_path`
`test_load_csv_devuelve_todas_las_filas`	Un CSV correcto se carga entero	`tmp_path`
`test_load_json_devuelve_todas_las_filas`	Un JSON se carga con `JsonSalesRepository`	`tmp_path`
`test_crear_repositorio_elige_clase_segun_extension`	`.csv` → `CsvSalesRepository`, `.json` → `JsonSalesRepository`	`isinstance`

`test_metrics.py` (4 tests)

Test	Qué comprueba
`test_importe_por_region_suma_y_ordena_de_mayor_a_menor`	Suma por región y orden descendente
`test_top_productos_devuelve_como_maximo_el_numero_configurado`	Como mucho `NUM_TOP_PRODUCTOS` productos, el primero el de más importe
`test_clientes_recurrentes_solo_incluye_mas_de_una_compra`	Solo los clientes con más de una compra
`test_resumen_calidad_cuenta_filas_y_suma_importe`	El diccionario de calidad completo

`test_integration.py` (2 tests, marca `integration`)

Test	Qué comprueba
`test_csv_curso_tiene_140_validas`	CSV real → 140 válidas
`test_csv_curso_tiene_10_invalidas`	CSV real → 10 inválidas

> **Sobre el nombre del test de integración:** el enunciado lo llama `test_csv_curso_tiene_ocho_validas`, pero la propia teoría de la sesión aclara que el checkpoint vigente de `ventas.csv` es **140 / 10** y que no se usen números de material anterior. Por eso se nombra según el resultado real.
> **`test_metrics.py`** no lo pide el enunciado de la E3, pero la estructura recomendada en la teoría lo incluye (`test_validator.py`, `test_loader.py`, `test_metrics.py`). Así cada módulo de `ventas_app` tiene sus tests.

---

2. Los tres conceptos clave (lo obligatorio de la sesión)

La teoría marca tres conceptos que hay que dominar al salir de clase:

```
AAA + assert     ->  "¿la función hace lo correcto?"
raises + params  ->  "¿falla donde debe? ¿y en varios bordes?"
fixture          ->  "¿reutilizo datos limpios sin copiar y pegar?"
```

2.1 Concepto 1 — Test unitario: patrón AAA y `assert`

Un **test unitario** comprueba una unidad pequeña (una función o un método) de forma **determinista**: con los mismos datos, siempre da el mismo resultado.

Patrón AAA:

1. **Arrange (preparar):** crear los datos de entrada.
2. **Act (actuar):** llamar a la función que se prueba.
3. **Assert (comprobar):** verificar el resultado.

Demo del curso (`2_pruebas_y_despliegue/ejemplos/test_calculator.py`):

```
from calculator import add

def test_add():
    # Arrange
    a, b = 2, 3
    # Act
    result = add(a, b)
    # Assert
    assert result == 5
```

Forma compacta, válida cuando es obvio: `assert add(2, 3) == 5`.

- `assert condición` no hace nada si es `True` y hace **fallar** el test si es `False`.
- pytest "reescribe" los `assert` para que, al fallar, muestren los valores reales: `assert 140 == 141` en vez de un simple "falso". Por eso en pytest se usa el `assert` normal de Python, sin métodos especiales.

2.2 Concepto 2 — Errores y bordes: `pytest.raises` y `parametrize`

`pytest.raises`: comprobar que el código falla bien

```
def test_divide_by_zero_raises():
    with pytest.raises(ZeroDivisionError):
        divide(1, 0)
```

- El test **pasa** si dentro del `with` se lanza esa excepción concreta.
- **Falla** si no se lanza ninguna, o si se lanza otra distinta.

`@pytest.mark.parametrize`: el mismo test con varios datos

Con un parámetro:

```
@pytest.mark.parametrize("precio, validas_esperadas", [(0, 0), (-1, 0), (1, 1)])
def test_validar_ventas_precio_limite(precio, validas_esperadas):
    ...
```

Con varios parámetros (demo `clamp` del curso), los nombres pueden ir como tupla:

```
@pytest.mark.parametrize(
    ("value", "low", "high", "expected"),
    [(5, 0, 10, 5), (-1, 0, 10, 0), (99, 0, 10, 10)],
)
def test_clamp(value, low, high, expected):
    assert clamp(value, low, high) == expected
```

- Se ejecuta **una vez por cada tupla** de la lista, y cada ejecución cuenta como un test.

- Evita copiar y pegar el mismo test cambiando solo los datos.

Por qué los bordes: los errores caros viven en los extremos: nulos, ceros, negativos, ficheros que no existen. Con la regla `precio > 0` se prueba justo el borde (`0`), justo fuera (`-1`) y justo dentro (`1`). Probar solo con `10` no detectaría un `>=` escrito por error. Si solo se prueba el "camino feliz", los tests dan una **falsa seguridad**.

2.3 Concepto 3 — Fixtures: datos reutilizables y aislados

```
@pytest.fixture
def ventas_mini():
    return pd.DataFrame({...})

def test_validar_ventas_mini_una_valida(ventas_mini):   # pytest la inyecta por
nombre
    validos, _ = validar_ventas(ventas_mini)
    assert len(validos) == 1
```

- Una **fixture** prepara datos o recursos y los **inyecta** en los tests que la piden como parámetro, por nombre.
- Se ejecuta **de nuevo para cada test**: cada test recibe datos limpios aunque otro los haya modificado.
- **Problema que resuelve:** sin fixture, el mismo `DataFrame(...)` se copiaría en 8 tests, y cambiar una columna obligaría a tocarlos todos. Con fixture se cambia **un** sitio.

Reglas prácticas de la teoría:

1. Los **unitarios** usan DataFrames **en memoria**, no el CSV del curso.
2. Si hacen falta ficheros, se usa `tmp_path` (ver 3.4).
3. No se comparte estado modificable entre tests: cada test debe poder ejecutarse solo.

Fixture o función auxiliar: en `test_validator.py` también hay `crear_venta(unidades, precio)`, una función normal. Se usa cuando cada test necesita **datos distintos**; la fixture devuelve siempre lo mismo.

---

3. Material suplementario

3.1 Pirámide de tests

```
       /\
      /E2E\         pocos, lentos, frágiles
     /------\
    / integ. \      algunos (CSV real, API simulada)
   /----------\
  /  unitarios \    muchos, rápidos, baratos
 /--------------\
```

Tipo	Qué prueba	Ejemplo	Analogía de la teoría
Unitario	Una función aislada con datos controlados	`validar_ventas(ventas_mini)`	Probar una bombilla fuera del edificio
Integración	Varias piezas juntas con I/O real acotado	`load` + `validar_ventas` con `ventas.csv`	Probar que el interruptor enciende la bombilla
End-to-end	El flujo completo como lo usaría el usuario	CLI → ficheros de salida (más adelante, API)	Recorrer la casa entera con un checklist

Heurística: la mayoría de los tests del Proyecto I deben ser **unitarios**.

3.2 Convenciones y nombres

- pytest busca ficheros `test_*.py` y, dentro, funciones que empiezan por `test_`. No hay que registrar nada.
- Los tests van en una carpeta **`tests/` separada**, nunca en el mismo fichero que el código.
- **Nombre recomendado:** `test_<unidad>_<escenario>_<resultado>`. Por ejemplo, `test_load_path_inexistente_lanza_data_load_error`: se prueba `load`, la ruta no existe y se espera `DataLoadError`. Si falla, el nombre ya explica qué se ha roto.

3.3 Leer la salida de pytest

Símbolo	Significado
`.`	Test pasado
`F`	Test fallido (un `assert` es falso)
`E`	Error inesperado (por ejemplo, una excepción dentro de una fixture)
`s`	Test saltado (*skipped*)

Parte 0 de la práctica: se rompe a propósito un valor esperado (por ejemplo, `VALIDAS_ESPERADAS = 141`). Al ejecutar `pytest -q` aparece:

```
>       assert len(validos) == VALIDAS_ESPERADAS
E       assert 140 == 141
FAILED tests/test_integration.py::test_csv_curso_tiene_140_validas - assert 140 ==
141
1 failed, 19 passed
```

El informe indica **qué test** falla, **en qué línea**, **qué valor se obtuvo** (140) y **cuál se esperaba** (141). Después se deja el valor correcto.

3.4 `tmp_path`: ficheros sin ensuciar el repo

```
def test_load_csv_devuelve_todas_las_filas(tmp_path):
    fichero = tmp_path / "ventas.csv"
    fichero.write_text("region,unidades\nNorte,2\nSur,3\n")
    assert len(load(fichero)) == 2
```

`tmp_path` es una fixture que trae pytest: crea una **carpeta temporal** para el test y la borra al terminar. Permite probar el loader sin tocar `Datos/`.

3.5 Marcas y filtro `-m`

```
@pytest.mark.integration
def test_csv_curso_tiene_140_validas():
    ...
```

Comando	Qué ejecuta
`pytest -q`	Todos los tests
`pytest -q -m "not integration"`	Solo unitarios (ideal mientras se programa)
`pytest -q -m integration`	Solo los de integración

- Las marcas propias **se registran** en `pytest.ini`; si no, pytest avisa con `PytestUnknownMarkWarning`.
- Marcas que trae pytest: `skip`, `skipif` (saltar si se cumple una condición), `xfail` (se espera que falle) y `parametrize`.

3.6 `pytest.ini`

```
[pytest]
pythonpath = .
testpaths = tests
markers =
    integration: tests que leen ficheros reales del curso (más lentos)
```

Opción	Qué hace
`pythonpath = .`	Añade la carpeta actual al path para poder hacer `from ventas_app import ...`
`testpaths = tests`	Busca los tests en `tests/`
`markers`	Registra las marcas propias

3.7 Comandos útiles

Comando	Qué hace
`pytest -q`	Salida resumida
`pytest -vv`	Muy detallada: nombre de cada test y diferencias completas
`pytest -q tests/test_loader.py`	Solo un fichero
`pytest -q tests/test_loader.py::test_load_path_inexistente_lanza_data_load_error`	**Un único test** (`fichero::nombre`)
`pytest -q --cov=ventas_app --cov-fail-under=60`	Cobertura (requiere `pip install pytest-cov`); falla si se cubre menos del 60 % del código

Cobertura (*coverage*): porcentaje de líneas del código que se ejecutan durante los tests. Es útil para encontrar zonas sin probar, pero un coverage alto **no** garantiza buenos tests: un test sin `assert` útil también "cubre" líneas.

3.8 Integración continua (CI)

- **CI** = ejecutar los tests **automáticamente** en un servidor en cada `push` o Pull Request.
- En GitHub se hace con **GitHub Actions**, en `.github/workflows/ci.yml` :

```
push / PR  ->  instalar dependencias  ->  pytest  ->  pasa o bloquea
```

- **Gate de merge:** si los tests fallan, la PR aparece en rojo y no se fusiona. Analogía de la teoría: CI es "el portero que no deja pasar una PR rota".
- **Por qué CI necesita tests deterministas:** si un test a veces pasa y a veces no (*flaky*), CI bloquea o deja pasar al azar y deja de ser fiable.
- **"Falla en CI y en local pasa":** suele deberse a versiones distintas, carpeta de ejecución distinta, marcas, ficheros no subidos a Git o un test flaky. Se reproduce lanzando **el mismo comando** que CI.
- La guía completa está en `2_pruebas_y_despliegue/02_ci_github.md` y el ejercicio **E4** (siguiente).

3.9 Anti-patrones (lista negra)

1. Tests que dependen del orden de ejecución.
2. Tests que necesitan internet sin motivo.
3. Un solo test gigante que lo prueba todo.
4. `assert True` o tests sin aserción real.
5. Capturar `Exception` en el código y no testear el error concreto.
6. Leer el CSV completo del curso en **todos** los unitarios.

7. Rutas absolutas (`C:/Users/...`) en los tests.
8. Cambiar el código solo para que el test pase, sin entender el fallo.
9. No fijar los datos de ejemplo (tests *flaky*).
10. Presumir de coverage alto con tests que no comprueban nada útil.

3.10 Por qué la arquitectura de la E2 hace posible testear

- En el script de la **E1**, todo se ejecuta de arriba abajo al importarlo: no se puede probar una parte sin ejecutarlo todo.
- En la **E2**, cada responsabilidad es una función de un módulo, y `validator` / `metrics` no tienen efectos laterales (I/O en los bordes). Eso permite importarlas, pasarles datos inventados y comprobar sus errores con `pytest.raises`.
- **Regla de oro de la teoría:** *si no puedes testearlo sin red, sin tu portátil y sin "Run All", aún no está lo bastante modular.*

3.11 Rúbrica de autocontrol (teoría)

Criterio	Insuficiente	Adecuado	Sólido
Conceptos clave	Solo `assert` suelto	AAA + raises + fixture	+ parametrize sistemático
Aislamiento	Depende de red o rutas propias	Fixtures en memoria	+ `tmp_path` / marcas
Bordes	Solo camino feliz	Errores tipados	Matriz de bordes clara
Proyecto I	< 8 tests	≥ 8 + 1 integration	README + CI local documentado

Esta práctica cumple el nivel **Sólido** en los tres primeros criterios.

---

4. Comandos usados

```
# Parte 0, en el repo del profesor (no se sube nada)
cd 2_pruebas_y_despliegue/ejemplos
pytest -q
pytest -vv
pytest -q test_calculator.py::test_divide_by_zero_raises

# En el repo personal, desde 1_programacion_avanzada_python
python -m pip install pytest
pytest -q
pytest -q -m "not integration"
```

---

5. Preguntas típicas de examen (E3)

1. **¿Qué es el patrón AAA?** Arrange (preparar datos), Act (llamar a la función) y Assert (comprobar el resultado).
2. **¿Cómo se prueba que una función lanza `ValueError`?** Con `with pytest.raises(ValueError):` y la llamada dentro del bloque.
3. **¿Para qué sirve `parametrize`?** Para ejecutar el mismo test con varios datos sin duplicar código; cada combinación cuenta como un test.
4. **¿Qué problema resuelve una fixture?** Evita repetir los mismos datos de prueba en muchos tests y da a cada test datos limpios e independientes.
5. **¿Unitario frente a integración, en una frase?** El unitario prueba una función aislada con datos en memoria; el de integración prueba varias piezas juntas con recursos reales (el CSV del curso).
6. **¿Qué ejecuta `pytest -m "not integration"`?** Todos los tests excepto los marcados como `integration`.
7. **¿Por qué CI necesita tests deterministas?** Porque un test que unas veces pasa y otras no hace que CI bloquee o deje pasar cambios al azar.
8. **¿Por qué probar los precios 0, -1 y 1?** Son los valores límite de la regla `precio > 0`; ahí es donde aparecen errores como escribir `>=` en vez de `>`.
9. **¿Qué es `tmp_path`?** Una fixture de pytest que crea una carpeta temporal y la borra al terminar el test.
10. **¿Cómo se ejecuta un solo test?** `pytest fichero.py::nombre_del_test`.
11. **¿Qué significan `.`, `F` y `E`?** Pasado, fallido (assert falso) y error inesperado.
12. **¿Por qué hay que registrar las marcas en `pytest.ini`?** Para que pytest las reconozca y no avise de marca desconocida.
13. **¿Se puede usar el CSV real en todos los tests?** No: el CSV real es para integración; los unitarios usan fixtures en memoria.
14. **¿Un coverage del 100 % garantiza que el código está bien?** No: solo indica qué líneas se ejecutaron, no que los `assert` comprueben algo útil.
15. **¿Qué es un test *flaky*?** Un test que da resultados distintos sin cambiar el código (por depender de la red, del reloj, del orden o de datos aleatorios sin semilla).
16. **¿Por qué el código modular (E2) es más fácil de testear que el script (E1)?** Porque cada función se puede importar y probar por separado con datos inventados; el script lo ejecuta todo al importarlo.
17. **¿Qué es una regresión?** Un fallo que aparece en algo que antes funcionaba, normalmente tras un cambio. Los tests la detectan en el momento.